

Informed search

Unit 3 · Intelligent Systems

School of Computer Science · Ciudad Real
Universidad de Castilla-La Mancha

Rubén Juárez Cádiz



The route through this unit

Concepts, working examples and laboratory decisions

Guide the search	Compare UCS, Greedy, A* and Weighted A* on one graph.
-------------------------	---

Justify the result	Distinguish admissibility, consistency and reopening.
---------------------------	---

Design a heuristic	Use the 8-puzzle, relaxed problems and pattern databases.
---------------------------	---

Build the lab solver	Derive and inspect bounds for geared Loopover.
-----------------------------	--

| By the end, a claim of optimality should come with explicit assumptions.

Known cost and estimated cost

$g(n)$

The accumulated cost of the path represented by node n .

$h(s)$

An estimate of the cheapest remaining cost from state s to a goal.

Two nodes can share a state and h value while having different paths and g values.

The heuristic uses the problem’s cost units

Objective	Suitable heuristic units	A modelling check
Fewest moves	Estimated remaining moves	One action must have a clear definition.
Shortest distance	Estimated remaining kilometres	Geometric distance needs a valid lower-bound argument.
Minimum travel time	Estimated remaining minutes	Distance alone is not a time estimate.

The goal and action model determine what h is allowed to mean.

Three ways to order the frontier

Strategy	Priority	What it favours
Uniform-cost search	$g(n)$	A cheap path so far.
Greedy best-first	$h(n)$	A state that appears close to a goal.
A*	$g(n) + h(n)$	A low estimated total cost via the node.

Setting $h = 0$ makes A* use the same cost ordering as UCS.

A frontier calculation

Candidate	g	h	g + h
A	2	6	8
B	4	2	6
C	5	5	10
D	7	1	8

Greedy chooses D

Its remaining-cost estimate is the smallest.

A* chooses B

Its total-cost estimate is the smallest.

Greedy best-first search

$$f = h$$

Remove the candidate with the smallest estimate of the remaining cost.

Potential benefit

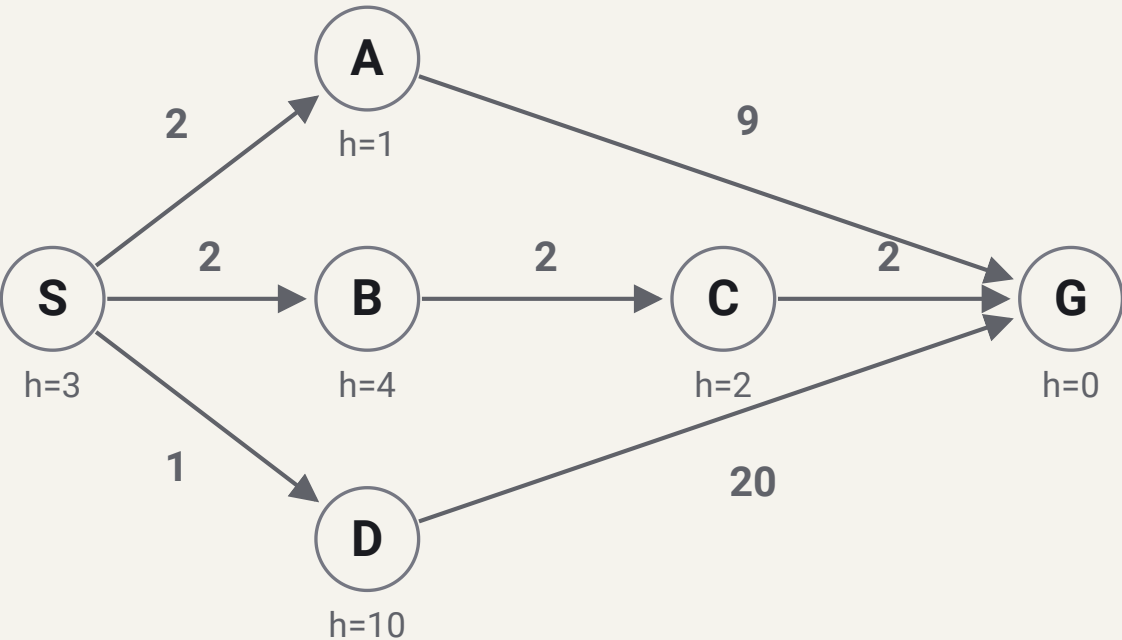
Useful guidance can lead to a goal after few expansions.

Limitation

The priority ignores the cost already paid.

Even an admissible heuristic does not make Greedy cost-optimal.

One graph with three competing routes



Route costs

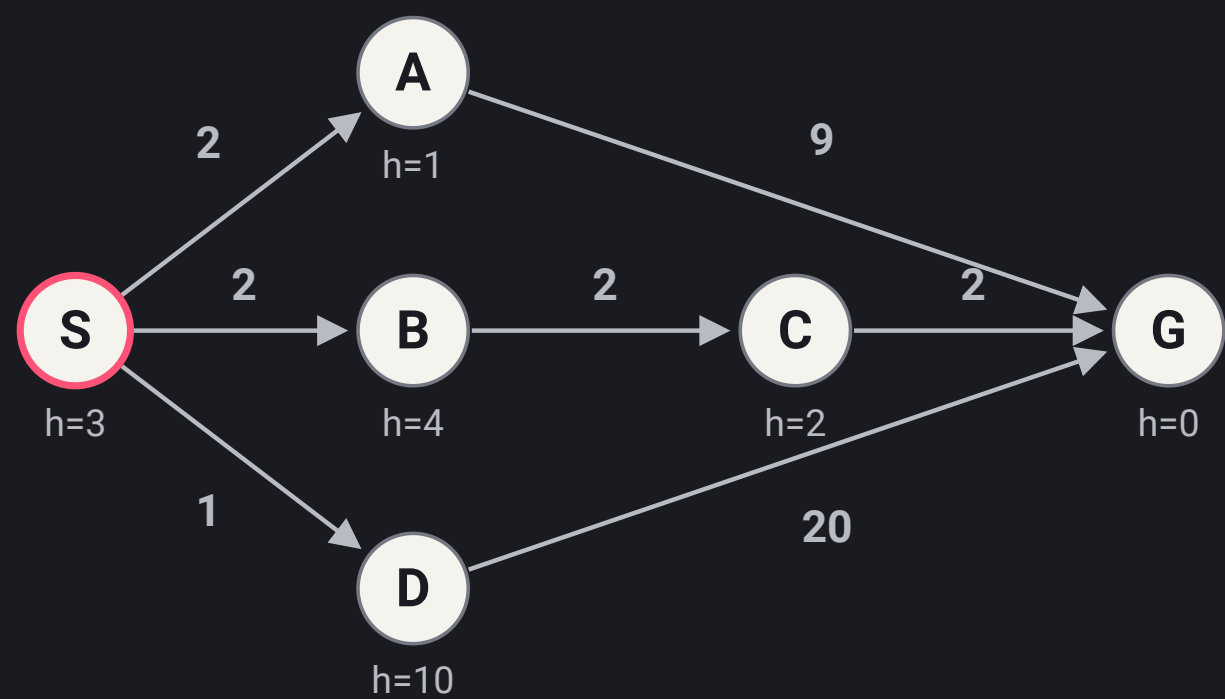
S, A, G: $2 + 9 = 11$
S, B, C, G: $2 + 2 + 2 = 6$
S, D, G: $1 + 20 = 21$

Heuristic values

S: 3 · A: 1 · B: 4
C: 2 · D: 10 · G: 0

Arrows define legal directions. Edge labels are costs; h labels are estimates.

UCS, Greedy and A* in action



Frontier priority

A* · g + h

▼

Weight w: 1

Select next

Run

Reset

Selection order

None selected

Frontier · next first · g / h / priority

S 0/3/3

Ready. Select the initial node S.

Ties: lower h, then insertion order. Goal test on removal. Best-g updates and stale-entry rejection.

Fewer expansions can buy a worse solution

Method	Selection order	Expanded nodes	Returned cost
UCS	S, D, A, B, C, G	5	6
Greedy	S, A, G	2	11
A*	S, A, B, C, G	4	6

These are exact counts for this graph and these tie rules, not general speed rankings.

Greedy guarantees depend on the setting

Setting	What follows
Finite reachable graph with duplicate control	Search eventually finds a reachable goal or exhausts the frontier.
Infinite search space	An infinite sequence of attractive states can prevent progress to a goal.
Unequal route costs	The first goal need not be cheapest, even with an admissible h.

Completeness and cost optimality are separate claims.

A* estimates a complete route through a node

$$f = g + h$$

Already paid + estimated remainder.

If h is admissible

f is a lower bound on a solution extending this particular path.

If h is zero

Only the accumulated cost affects the primary order.

The estimate guides which candidate to examine. The returned path still needs validation.

Graph A* with best-cost improvements

```
best[start] ← 0
push(root(start), priority=h(start))
while frontier is not empty:
    n ← pop_min_f()
    if n.g != best[n.state]: continue
    if goal(n.state): return reconstruct(n)
    for (a, s, cost) in successors(n.state):
        new_g ← n.g + cost
        if new_g < best.get(s, infinity):
            best[s] ← new_g
            push(Node(s, n, a, new_g), new_g + h(s))
return FAILURE
```

A cheaper path can be queued even when the state was expanded earlier.

Queue entries and state records have different jobs

best[state]

The smallest g discovered so far for that state.

Queue entry

An immutable candidate path with its own g, parent and priority.

Stale entry

Its g no longer matches best[state], so it is skipped.

Reopening

A cheaper path reaches a state whose successors were already explored.

The presence of a state in a set does not tell you whether its current path is cheapest.

Admissibility is a global lower-bound claim

$$0 \leq h(s) \leq h^*(s)$$

$h^*(s)$

The actual minimum remaining cost to any goal.

Consequences

Underestimation is allowed.

Every goal has $h = 0$.

Checking several examples can disprove admissibility, but cannot establish it for every state.

Consistency constrains every legal step

$$h(s) \leq c(s, a, s') + h(s')$$

Local interpretation

The estimate cannot drop by more than the cost of taking one step.

Goal boundary

Set $h(\text{goal}) = 0$.

The inequality applies to every directed transition.

Consistency plus the goal boundary implies admissibility for states that can reach a goal.

Consistent heuristics make f nondecreasing

$$\begin{aligned} f(n') &= g(n) + c(s,a,s') + h(s') \\ &\geq g(n) + h(s) = f(n) \end{aligned}$$

Along a generated path

The estimated total cost never falls.

For ordinary A*

A non-stale state removed at minimum f already has its cheapest g .

With a consistent heuristic, an expanded state does not need reopening.

An interactive heuristic check

Edges: $S \rightarrow A:3$, $S \rightarrow B:1$, $B \rightarrow A:1$,
 $A \rightarrow G:3$, $B \rightarrow G:10$.

h(S): 5

h(A): 0

h(B): 4

$h(G) = 0$. The graph is finite and all its states are shown.

Consistent values

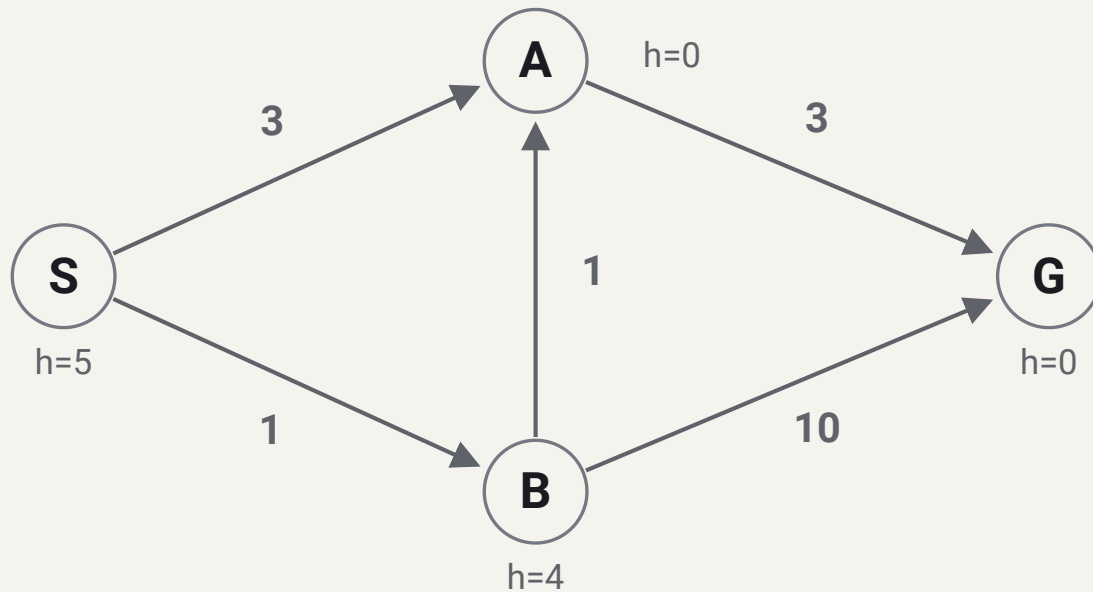
Reset

State	h	True h*	Lower bound?
S	5	5	Yes
A	0	3	Yes
B	4	4	Yes
G	0	0	Yes

Admissible: YES. Consistent: NO.

Failed edges: $S \rightarrow A$ ($5 > 3$); $B \rightarrow A$ ($4 > 1$).

Admissible heuristics may require reopening



Two prefixes to A

$S \rightarrow A$ costs 3.

$S \rightarrow B \rightarrow A$ costs 2.

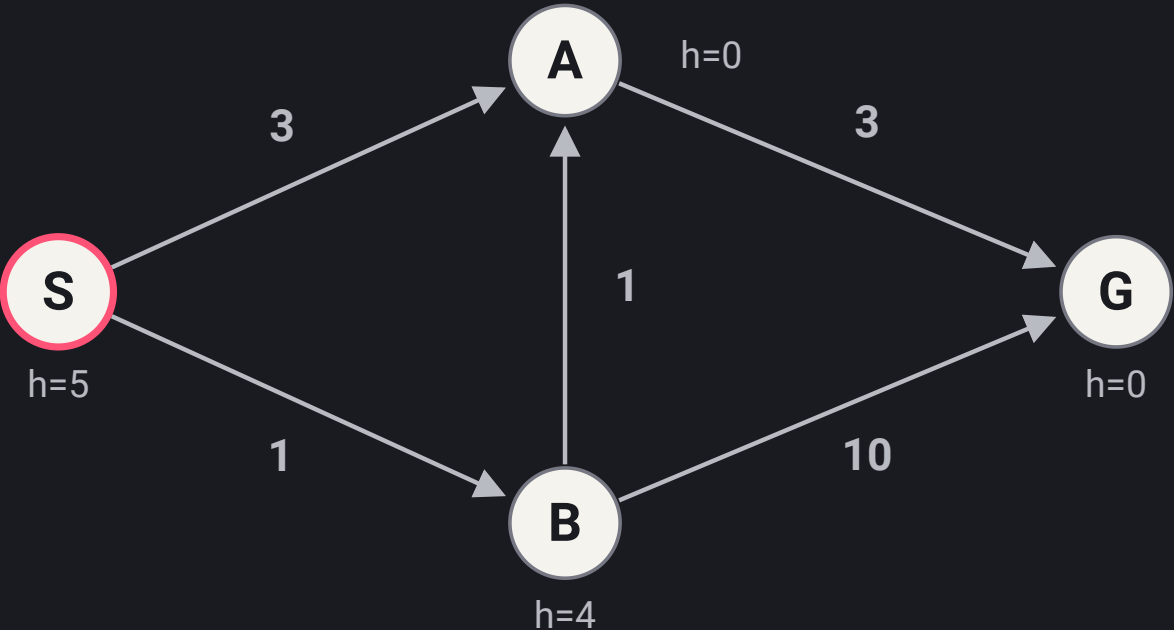
Fixed heuristic

$h(S)=5$, $h(A)=0$,

$h(B)=4$, $h(G)=0$.

A is expanded at $g = 3$ before B reveals its cheaper route.

Reopening changes the returned answer



☒ Allow expanded states to reopen

Select next

Run

Reset

Selection order

None selected

Frontier · state and g / h / f

S 0/5/5

Ready. Select the initial node S.

Fixed $h = \{S:5, A:0, B:4, G:0\}$. The toggle deliberately demonstrates an unsafe closed-set policy.

The decisive improvement

Event	$g(A)$	Best goal candidate	Meaning
A expanded via S	3	G at cost 6	The first route is expensive.
B offers a better route	2	Still G at cost 6	A needs another expansion.
A reopened via B	2	G at cost 5	The goal path improves.
Non-stale G selected	—	Return cost 5	Old G(6) is irrelevant.

A revised parent chain is part of the improvement.

Why an expensive goal must wait

Candidate goal G

Its heuristic is zero, so $f(G)=g(G)$.

Suppose G is suboptimal

Then $g(G)$ is greater than the optimal cost C^* .

Candidate on an optimal path

With admissibility and correct path improvements, some frontier node n satisfies $f(n) \leq C^*$.

Priority consequence

n is selected before the expensive goal.

The argument needs a correct frontier, an admissible h and appropriate handling of repeats.

Conditions for A* guarantees

Setting	Cost guarantee	Termination condition
Tree A* with admissible h	First selected goal is optimal.	For standard completeness: finite branching, finite solution cost and $c \geq \epsilon > 0$.
Graph A* with consistent h	Optimal without reopening closed states.	Finite reachable graph, or the usual positive-cost infinite-space assumptions.
Graph A* with admissible, inconsistent h	Optimal with correct improvements and reopening.	The same distinction between finite and infinite spaces matters.

Nonnegative costs alone do not rule out an infinite zero-cost search region.

Tie-breaking changes the amount of work

Primary order

Always choose a node with minimum f .

Secondary order

Lower h , then insertion sequence, is the convention used here.

**Equal $f \Rightarrow$ larger g
means smaller h**

Those two tie rules are equivalent when $f = g + h$ and the primary f value is equal.

A deterministic rule makes traces reproducible. It does not make every run equally efficient.

Dominance compares lower bounds

Condition	Meaning
h_1 and h_2 are admissible	Both remain below the same true remaining cost.
$h_2(s) \geq h_1(s)$ for every state	h_2 dominates h_1 and provides an equally strong or stronger bound.
Evaluation cost also matters	A stronger estimate can take longer to compute.

Dominance alone does not promise a shorter runtime for every implementation and instance.

Combining heuristics with a maximum

$$h(s) = \max(h_1(s), h_2(s))$$

If both estimates are lower bounds, their maximum remains a lower bound.

Consistency is preserved too

When each component is consistent under the same action costs, their maximum is consistent.

A sum needs a further argument

Two estimates may count the same paid action.

Two heuristics can each equal 1 while only one action is needed. Their sum would overestimate.

A heuristic needs an explicit goal board

Supplied starting board

2	8	3
1	6	4
7	□	5

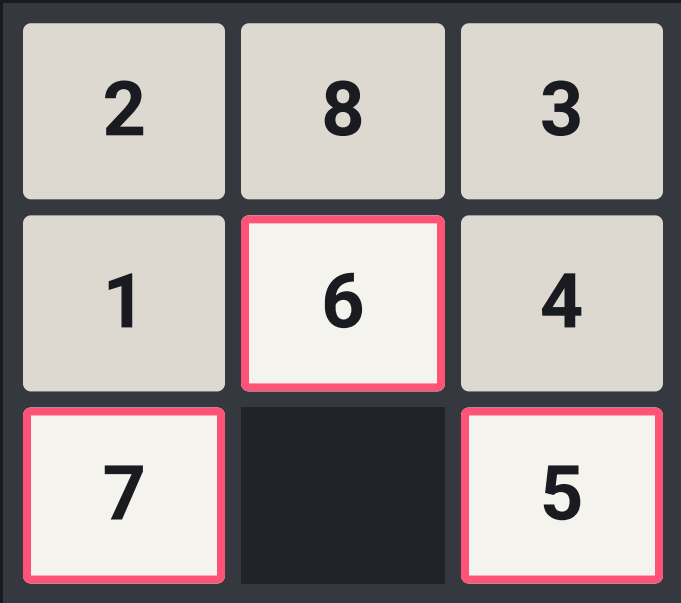
Goal used in this example

1	2	3
8	□	4
7	6	5

Unit cost per legal blank move. The blank is excluded from both heuristic sums.

Move a tile and inspect both heuristics

Click a numbered tile next to the blank.



Source example

One move away

Goal

Misplaced numbered tiles

4

Sum of Manhattan tile distances

5

Both values are lower bounds on remaining moves.
Manual moves: 0.

Goal: 1 2 3 / 8 □ 4 / 7 6 5.
One legal move costs one.

Why Manhattan dominates misplaced tiles

Each misplaced numbered tile	Contributes at least one unit to Manhattan distance.
One legal blank move	Moves exactly one numbered tile by one grid step.
Lower-bound argument	One action can reduce the total Manhattan distance by at most one.
Consequence	$h_{\text{misplaced}} \leq h_{\text{Manhattan}} \leq$ the optimal number of moves.

The argument relies on this puzzle's movement and cost rules.

A five-move certificate

Move	Blank direction	Numbered tile moved
1	Up	6
2	Up	8
3	Left	2
4	Down	1
5	Right	8

Upper bound

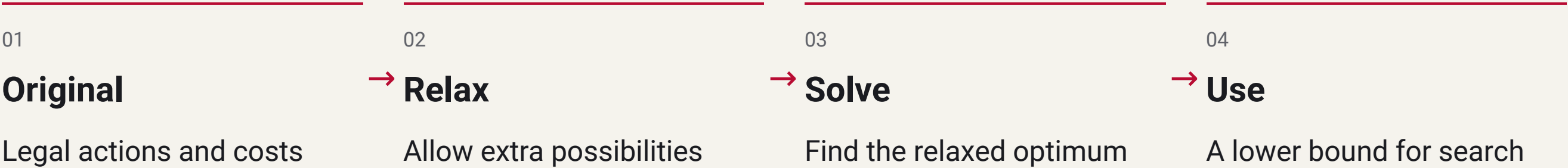
The legal five-move sequence reaches the displayed goal.

Lower bound

Initial Manhattan distance is 5.

A cost-5 plan and a cost-5 lower bound together prove that the plan is optimal.

Relaxation removes constraints



Required relationship

Every concrete plan must have a relaxed counterpart costing no more.

Failure mode

An altered problem that makes actions more expensive need not give a lower bound.

The exact cost of a valid relaxation cannot exceed the original optimum.

Exact abstract costs can guide concrete search

Concrete state

All tile identities and positions.

Abstract state

Positions of selected tiles. Other identities are ignored.

Stored value

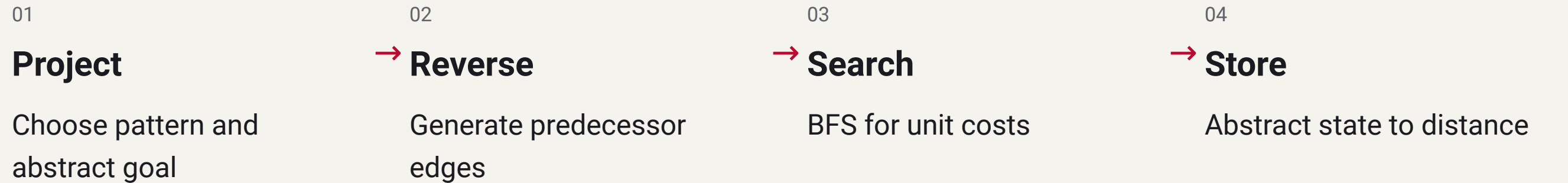
The cheapest abstract cost to the abstract goal.

Lookup

Map the concrete state to its pattern and retrieve the value.

Every concrete action must map to an abstract transition of no greater cost.

A database of distances to the goal



Reverse Dijkstra is the corresponding choice for general nonnegative edge costs.

More tracked tiles require more storage

Pattern tiles k	Distinct position assignments $16! / (16-k)!$	Packed values at 1 byte each
2	240	240 bytes
4	43,680	42.66 KiB
6	5,765,760	5.50 MiB
8	518,918,400	494.88 MiB

These are indexing counts and illustrative packed-value sizes, not measured Java memory use.

Adding patterns requires a cost budget

Safe maximum

$\max(\text{PDB}_1, \text{PDB}_2)$ preserves admissibility when both are lower bounds.

Potentially unsafe sum

Disjoint sets of tile labels can still benefit from the same action.

Cost partitioning

$$c_i(a) \geq 0$$

$$\sum_i c_i(a) \leq c(a)$$

Build each PDB using its assigned action costs before summing.

In coupled Loopover, one action moves several tiles, possibly from several patterns.

The coupled action is the cost unit

00	01	02	03
04	05	06	07
08	09	10	11
12	13	14	15

State

A full 4 × 4 board of distinct tiles 00–15.

Action r c +

Shift row r right, then column c down.

Action r c −

Shift row r left, then column c up.

Cost

One per complete coupled action.

Column c refers to the fixed column index after the row phase.

One action moves seven distinct tiles

Support of the transformation

Four row positions and four column positions share one cell.

$$4 + 4 - 1 = 7$$

Total elementary displacement

The row phase moves four tiles one step. The column phase moves four tiles one step.

$$4 + 4 = 8$$

One tile participates in both phases. Distinct tiles and unit displacements are different counts.

Raw tile counts can overestimate the cost

One move before the goal

01	06	03	00
04	05	10	07
08	09	14	11
12	13	02	15

Apply 02+

The row-right then column-down action reaches the ordered goal.

Before the action

Misplaced tiles $m = 7$

Toroidal distance sum $D = 8$

True remaining cost = 1

Neither raw m nor raw D is admissible under this coupled unit-cost model.

A lower bound from misplaced tiles

$$h_m(s) = \lceil m(s) / 7 \rceil$$

At most seven repairs One action affects seven distinct tile positions.

Any solution of length L Can repair at most 7L currently misplaced tiles in total.

Lower bound $m(s) \leq 7L$, so every solution needs at least $\lceil m/7 \rceil$ actions.

The ceiling matters because the action count is an integer.

A lower bound from toroidal distance

$$D(s) = \sum \text{tiles } [\min(|\Delta r|, 4-|\Delta r|) \\ + \min(|\Delta c|, 4-|\Delta c|)] \\ hD(s) = \lceil D(s) / 8 \rceil$$

Wraparound distance

Row 0 and row 3 are one step apart on a four-cell ring.

Per-action progress

The eight elementary displacements can reduce D by at most eight.

$D(s) \leq 8 + D(s')$ also gives consistency of the rounded bound.

A combined bound with a clear proof

$$h_{\text{basic}} = \max(h_m, h_D)$$

Both components are admissible and consistent.

Adding a pattern database

$$h = \max(h_m, h_D, h_{\text{PDB}})$$

The same properties hold for a cost-preserving PDB.

Using $h_m + h_D$ would already overestimate on the one-move example: $1 + 1 > 1$.

A live Loopover heuristic laboratory

01	06	03	00
04	05	10	07
08	09	14	11
12	13	02	15

01060300040510070809141112130215

Crimson tiles 00 and 01 form the pattern.

Verified example

One-move certificate ▾

Row

0 ▾

Column

2 ▾

Sign

+ ▾

Apply action

Undo

Reset example

Raw misplaced m

7

Raw distance D

8

[m / 7]

1

[D / 8]

1

Two-tile PDB

1

Combined maximum

1

Optimal plan: 02+. Cost 1. PDB: 240 states.

A pattern detects what counts miss

12	01	02	00
04	05	06	07
08	09	10	11
03	13	14	15

Only three tiles are misplaced

$m = 3, D = 4.$

$h_m = 1$ and $h_D = 1.$

The two-tile abstraction says more

$h_{PDB} = 2.$

A matching solution

00+ followed by 00- reaches the goal.

A two-step lower bound plus a two-step solution proves the optimum is 2.

Reverse generation must reverse phase order

Forward 02+

1. Row 0 right.
2. Column 2 down.

Predecessor transformation

1. Column 2 up.
2. Row 0 left.

The predecessor operation is for reverse search. It does not introduce a new legal forward action.

Three exact solution certificates

Example	m / D	h _m	h _D	h_PDB	Optimal cost
One-move predecessor	7 / 8	1	1	1	1
Three displaced tiles	3 / 4	1	1	2	2
Three-action predecessor	15 / 22	3	3	2	3

Each optimum follows from a matching lower bound and a verified legal plan.

Weighted A* changes the priority

$$f_w = g + w h$$

At $w = 1$

The priority is ordinary A*.

At $w > 1$

The heuristic contributes more to the priority.

On the six-state example

Weight	Returned cost	Expanded
1	6	4
2	6	4
3	11	2

A larger weight can change the route. Fewer expansions are not guaranteed on every problem.

The Weighted A* cost bound

$$C_{\text{returned}} \leq w C^*$$

For fixed $w \geq 1$ with an admissible nonnegative h and $h(\text{goal})=0$.

Implementation conditions

Nonnegative action costs, minimum-priority removal, valid best-g improvements and reopening when required.

Termination

A finite reachable graph suffices here. Infinite spaces need further completeness assumptions.

An arbitrary overestimating heuristic does not inherit this weight bound.

IDA* deepens an f threshold

01

Initialize

threshold = $h(\text{start})$

02

→ Traverse

Depth-first within $g + h \leq$
threshold

03

→ Record

Smallest exceeded f value

04

→ Repeat

Use that value as the next
threshold

Memory advantage

Store the current path and limited traversal
information.

Work repeated

The search regenerates states across thresholds.

IDS deepens a depth bound. IDA* deepens a cost-estimate bound.

Other ways to limit search memory

Method	Main idea	What it gives up or needs
RBFS	Follow the most promising path and retain backed-up alternatives.	May regenerate forgotten subtrees.
SMA*	Use a memory budget and forget poor leaves while backing up their values.	Guarantees depend on enough memory for relevant solution paths.
Full A*	Retain frontier candidates and cost records.	Memory may grow too large before the goal is selected.

A memory limit is a reported search outcome, not proof that a puzzle is unsolvable.

A useful benchmark isolates the heuristic

Same task set	Identical initial states, goals, action costs and successor order.
Same correctness policy	Best-g updates, reopening, stopping rule and tie breaker.
Search effort	Generated nodes, expansions, reexpansions and peak frontier.
Actual cost	Wall time, heuristic time, peak memory and returned solution cost.

Count the PDB build cost and storage, and state whether they are amortized across many runs.

Responsibilities carried over from Unit 2

Course artefact	Responsibility in informed search
Estado	Immutable board and equality by configuration.
Sucesor	Action, resulting state and step cost.
Nodo	Parent, action, g, depth and computed h or f.
Frontera	Minimum-priority removal with deterministic ties.
Heurísticas / HeurísticasPBD	State-based analytical bounds and pattern lookup.
Solver	Improvements, stale checks, reopening and path reconstruction.

The class names follow the supplied slides. Match exact filenames to the actual lab repository.

The A* implementation contract

Immutable queue records	Changing an enqueued priority in place can invalidate the heap order.
--------------------------------	---

State-keyed best cost	Equality identifies configurations, independent of parent or depth.
------------------------------	---

Heuristic on the child	Compute h from the resulting state and the correct goal.
-------------------------------	--

Revised path	A cheaper g comes with the corresponding parent and action.
---------------------	---

Honest termination	Solution, frontier exhausted, or an explicit resource limit.
---------------------------	--

| The implementation and the proof must describe the same algorithm.

Tests with known answers

Test	Expected evidence
Goal input	Empty plan, cost 0 and h=0.
Six-state graph	A*:6, UCS:6, Greedy:11 under the stated rules.
Inconsistent counterexample	A* with reopening returns 5.
One-move Loopover predecessor	02+ reaches the goal; derived bounds equal 1.
Two-tile abstract graph	240 reachable entries; h(goal)=0; every edge passes consistency.
Parent and inverse checks	No mutation of input; true inverse restores the board.

A finite sample can catch heuristic errors. A general claim still needs a proof.

Workshop: trace A* and explain the stopping rule

8 min

Use the six-state graph from the search laboratory.

Primary priority $g+h$.

Ties: lower h , then insertion.

After S

List A, B and D with g , h and f .

After A

Insert G and choose the next candidate.

After B and C

Update the goal path and identify stale entries.

Explain

Why is the first generated goal not accepted?

Deliver a selection trace, a parent-based solution path and the final cost.

Workshop solution

After selecting	Active frontier in priority order	Update
S	A:2/1/3, B:2/4/6, D:1/10/11	Numbers are g / h / f.
A	B:2/4/6, G:11/0/11, D:1/10/11	An expensive goal is discovered.
B	C:4/2/6, G:11/0/11, D:1/10/11	C extends the cheaper route.
C	G:6/0/6, D:1/10/11	G(11) becomes stale.
G	Stop with cost 6	Path $S \rightarrow B \rightarrow C \rightarrow G$.

A stays in the selection trace, but it does not appear in the returned solution path.

Workshop: defend or reject a heuristic

Proposed heuristic	Question to justify
h=0 on the entire state space	What does it guarantee, and how much guidance does it provide?
2 × Manhattan for the unit-cost 8-puzzle	Can one nearby state refute admissibility?
Raw misplaced tiles for coupled Loopover	How many tiles can one paid action repair?
Sum of two full-cost disjoint-tile PDBs	Can one action help both patterns?

For a rejection, provide a counterexample. For acceptance, state a general argument.

Heuristic workshop: reasoned answers

Proposal	Conclusion and evidence
h=0	Admissible and consistent for nonnegative costs; A* becomes UCS.
2 × Manhattan	Not generally admissible: one move away gives 2 instead of 1.
Raw misplaced Loopover tiles	Not admissible here: the verified one-move state has 7 misplaced tiles.
Sum of full-cost PDBs	Not guaranteed: an action can be counted in both estimates. Use a maximum or prove a cost partition.

“Admissible” always refers to a particular problem, goal and cost model.

Six checks for optimal A*

01

What does g describe?

The exact cost of this candidate path, not necessarily the cheapest path to its state.

02

What does admissibility require?

A lower bound on remaining cost for every relevant state.

03

What does consistency add?

An inequality for every legal edge, with $h(\text{goal})=0$.

04

When might A* reopen a state?

When an inconsistent heuristic lets it expand a state before a cheaper path arrives.

05

Why can a raw Loopover count fail?

One coupled unit-cost action moves seven distinct tiles.

06

When can PDB values be added?

When a valid cost allocation prevents counting the same action cost more than once.

Sources and further reading

Supplied course deck

01_Presentacion_Tema3_Informed_Search_Algorithms(3).pptx, 42 slides. Scope, core definitions, 8-puzzle starting board and lab terminology.

Russell and Norvig · Artificial Intelligence: A Modern Approach

[Official textbook resources](#) · [UC Berkeley CS188: Informed Search](#)

Felner, Korf and Hanan (2004)

Additive Pattern Database Heuristics, JAIR 22, 279–318.

[Author-hosted paper](#)

Hansen and Zhou (2007)

Anytime Heuristic Search, JAIR 28, 267–297.

[Paper and Weighted A* discussion](#)

Constraints can change the search representation

Unit 3

Configurations, legal transitions and heuristic estimates.

$g + h$

Unit 4

Variables, domains and constraints.

$X \cdot D \cdot C$

The next question: which choices can be ruled out before building a complete assignment?